



A fully hardware-managed scheduling architecture for AI accelerators

Libo Cheng¹ · Liang Yang¹ · Jian Shao¹ · Xinyi Gu¹ · Rong Qian¹ · Xinwei Zhang¹ · Donghao Li¹ · Hongbin Wang¹ · Xiaoqi Xia¹

Received: 23 November 2025 / Accepted: 22 January 2026 / Published online: 5 February 2026
© The Author(s) 2026

Abstract

The proliferation of AI applications across diverse domains has driven the evolution of AI accelerators toward higher performance and energy efficiency. This paper addresses the critical challenge of task scheduling in AI accelerators by introducing a fully hardware-managed scheduling system. Our approach leverages Operator Completion Status Registers (OCSRs) and a novel computation-scheduling instruction set to minimize software overhead and maximize execution parallelism. The co-designed hardware-software solution comprises: (1) a dedicated hardware scheduling unit with a complete instruction pipeline, (2) a compiler that maps operators to scheduling instructions while managing OCSR allocation, and (3) a lightweight runtime for efficient task dispatch. Experimental results demonstrate that our system significantly reduces scheduling latency and improves overall throughput, achieving an average performance gain of approximately 30% across multiple CNN models while maintaining minimal area overhead of only 7.41%. The proposed architecture establishes a new paradigm for high-efficiency AI accelerator design.

Keywords AI accelerators · Hardware-managed scheduling · OCSR · Computation-scheduling instructions · Scheduling latency · Throughput optimization · Neural network inference

✉ Libo Cheng
chenglb09@163.com

✉ Liang Yang
yangliang1110@gmail.com

Jian Shao
shaojian1008@126.com

Xinyi Gu
briskthrough@163.com

Rong Qian
qianrong0316@126.com

Xinwei Zhang
powerxinwei@163.com

Donghao Li
lli471426@gmail.com

Hongbin Wang
whbwyf@qq.com

Xiaoqi Xia
natsuxiaxq@163.com

¹ China Electronics Technology Group Corporation 58th Research Institute, Wuxi 214100, China

1 Introduction

Artificial Intelligence (AI) has emerged as a transformative technology, reshaping methodologies and capabilities across numerous domains including healthcare, autonomous systems, financial services, and industrial automation. Recent advances in machine learning, deep learning, natural language processing, and computer vision have enabled AI systems to achieve unprecedented performance in critical tasks such as pattern recognition (Lu et al. 2024b), predictive analytics (Dixon et al. 2024), and autonomous systems (Lu et al. 2024a). The exponential growth in model complexity and data volumes has created unprecedented computational demands, necessitating specialized hardware solutions that can deliver both high throughput and energy efficiency.

To enhance the inference performance of trained models in real-world deployments, substantial research efforts have focused on three primary directions: algorithmic optimization (Zhou et al. 2018), software compilation techniques (Chen et al. 2018; Golin et al. 2024), and specialized hardware acceleration. While algorithmic improvements

provide theoretical benefits, and compilation techniques optimize software execution, it is the hardware acceleration that ultimately determines the practical performance boundaries of AI systems in production environments.

While conventional processors, including CPUs and GPUs, remain adequate for general-purpose computing, they fundamentally lack the architectural specialization required to execute AI workloads efficiently. This performance-energy efficiency dichotomy has catalyzed the rapid evolution of dedicated AI accelerator chips, transforming them from theoretical concepts into indispensable components of the AI revolution. The growing gap between computational demands of state-of-the-art models and the capabilities of general-purpose hardware has created an urgent need for domain-specific architectures optimized for neural network computations.

Recent years have witnessed remarkable advances in AI chip capabilities across multiple dimensions. Through innovations in parallel processing architectures (Chen et al. 2014; Jouppi et al. 2017), near-memory computation (Verma et al. 2019), and advanced fabrication processes (Li et al. 2024; Chen et al. 2019), modern AI accelerators deliver orders-of-magnitude improvements in both computational throughput and operations-per-watt efficiency. Industry leaders (e.g., TPUs, NPU, and neuromorphic chips) alongside academic breakthroughs continue to push the boundaries of intelligent hardware design. However, most of these advancements have focused primarily on computational units and memory hierarchies, often overlooking the critical role of efficient task scheduling in overall system performance.

Although current research predominantly focuses on enhancing computational (Malathi et al. 2024) and memory units (Tam et al. 2020; Gholami et al. 2024; Wang et al. 2024), task scheduling—a pivotal factor in overall system efficiency—remains relatively unexplored despite its significant impact on real-world performance. Efficient scheduling strategies can optimize computational resource utilization, reduce latency, hide memory access overhead, and significantly improve effective computation capability. In many practical deployments, inefficient scheduling can undermine the theoretical advantages of powerful computational units, leading to substantial underutilization of hardware resources.

To address this critical gap, we propose an innovative hardware-based scheduling solution designed around task scheduling optimization objectives. Our approach aims to minimize software scheduling overhead while enhancing task execution parallelism and real-time responsiveness. The co-designed hardware-software system represents a fundamental shift from traditional software-dominated scheduling paradigms to a hardware-managed approach that can fully

leverage the parallel execution capabilities of modern AI accelerators. The system features three key components:

- **Architecture:** A computation-scheduling instruction set employing Operator Completion Status Registers (OCSR) as source/destination registers to establish explicit operator dependencies and enable fine-grained parallelism.
- **Hardware Implementation:** A dedicated execution pipeline (fetch-decode-issue-execute-writeback) for scheduling instructions, enabling parallel operator execution through multi-issue support and out-of-order execution capabilities.
- **Software Stack:** Virtual OCSR allocation/reclamation mechanisms, computation-scheduling instruction mapping for model compilation and code generation, and a lightweight runtime system that minimizes host processor involvement.

Experimental results validate the superior performance of the proposed hardware scheduler across a range of AI workloads. In comparison to conventional software-based schedulers, the proposed design significantly reduces scheduling latency and enhances effective chip throughput, setting new benchmarks in accelerator efficiency. These improvements are achieved with minimal area overhead, rendering the architecture particularly suitable for AI applications where computational efficiency and power consumption are critical.

2 Background and related work

As AI technology permeates various industries and application scenarios, AI chips have evolved with distinct architectural emphases tailored to specific deployment environments and computational patterns. From an application perspective, AI chips can be broadly categorized into cloud-side deployments (e.g., recommendation systems for e-commerce platforms, city-scale AI systems such as “City Brains,” and large language model inference servers) and edge-side applications (e.g., smartphones, security surveillance, autonomous driving, and IoT devices). Each category presents unique challenges: cloud-side deployments prioritize high throughput and scalability, while edge-side applications emphasize power efficiency, latency sensitivity, and cost constraints.

Architecturally, AI accelerators can be classified into several distinct types based on their computational paradigms and execution models: Instruction Set Processors (e.g., Cambricon MLU series products Liu et al. 2016) that extend traditional processor architectures with AI-specific instructions, Dataflow Processors (Chen et al. 2019) that optimize for data movement patterns in neural networks, In-Memory

Computing Processors (Halawani et al. 2021) that reduce data transfer overhead through processing-in-memory techniques, and Reconfigurable AI Processors (Li et al. 2024) that provide flexible computational fabrics adaptable to varying model structures. Each architectural approach represents different trade-offs between flexibility, efficiency, and programmability.

Our proposed fully hardware-scheduled system primarily targets edge-side applications and employs a streaming architecture for AI chips, focusing on minimizing control overhead and maximizing hardware utilization. The NVIDIA Deep Learning Accelerator (NVDLA) serves as a representative architecture for this approach due to its open-source nature, modular design, and widespread adoption in both academic research and industrial applications. This section examines both NVDLA's architecture and its software scheduling workflow to establish the baseline against which our improvements are measured.

2.1 NVDLA architecture overview

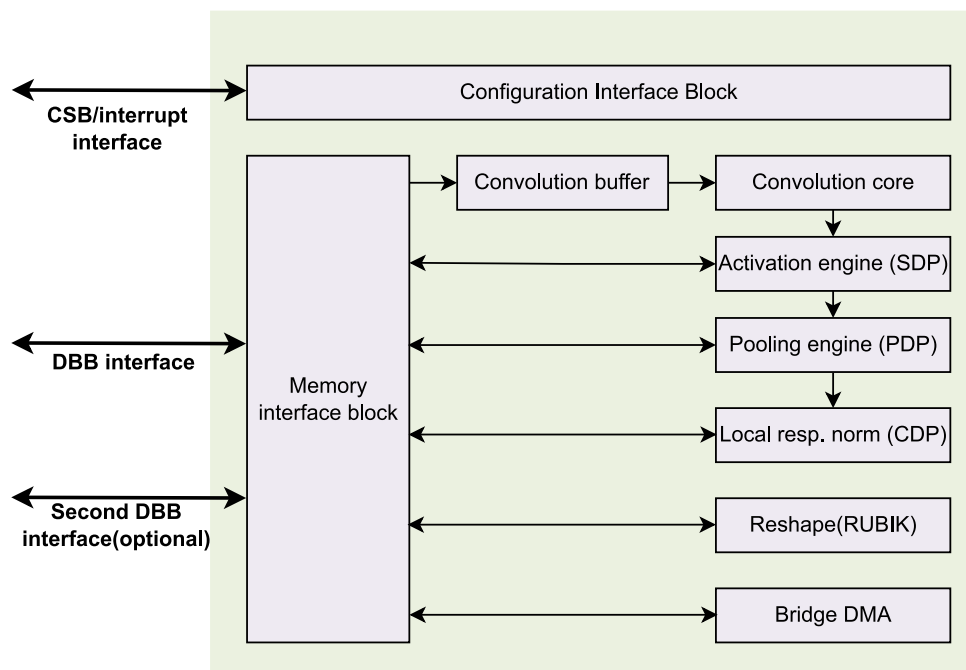
NVDLA represents an open-source, modular hardware architecture designed to standardize and accelerate deep learning inference workloads (Nvdla hardware architectural specification 2017). Released by NVIDIA in 2017, it provides a scalable framework that bridges the gap between computational demands of modern neural networks and resource constraints across diverse platforms—ranging from cost-sensitive IoT devices to high-performance edge systems. The architecture's modularity enables customization for specific

application requirements while maintaining software compatibility through a consistent programming interface.

The NVDLA architecture, depicted in Fig. 1, incorporates several specialized processing units (Nvdla unit description 2017) that work in concert to execute complete neural network models:

- **Convolution Core:** Optimized for high-performance convolutions utilizing Winograd transforms and sparse weight compression techniques. The core employs sophisticated data reuse strategies to minimize external memory accesses and maximize computational efficiency for the most common operation in convolutional neural networks.
- **Single Data Processor (SDP):** Executes activation functions (ReLU, sigmoid, tanh) and element-wise operations including addition, multiplication, and scaling. The SDP handles operations that require per-element processing rather than spatial or channel-wise transformations.
- **Planar Data Processor (PDP):** Handles 2D pooling operations (max/average pooling) for spatial dimension reduction. The processor efficiently implements various pooling window sizes and stride configurations common in modern network architectures.
- **Channel Data Processor (CDP):** Manages channel-wise normalization operations (e.g., Local Response Normalization, batch normalization) that are essential for training stability and inference accuracy in deep networks.
- **Reshape Engines (RUBIK/BDMA):** Accelerate tensor transformations (transpose, concatenation, splitting) and

Fig. 1 NVDLA block diagram illustrating key computational units and memory interfaces. The architecture features specialized processors for different types of neural network operations



memory transfer operations. These units handle data layout transformations that are frequently required between different types of neural network layers.

To address bandwidth constraints that often bottleneck AI accelerator performance, NVDLA implements a sophisticated heterogeneous memory strategy:

- **Primary DBB Interface:** Connects to system DRAM shared with CPU and I/O subsystems, providing large-capacity storage for model parameters, input data, and intermediate activations that exceed on-chip memory capacity.
- **Secondary DBB Interface (optional):** Dedicated to high-bandwidth SRAM, significantly reducing latency for weight and activation caching in large model deployments. This interface enables efficient data reuse patterns and reduces contention with other system components.

The combination of specialized processing units and a hierarchical memory architecture enables NVDLA to achieve high computational density while maintaining flexibility across diverse neural network models and deployment scenarios.

2.2 Software scheduling in NVDLA

The NVDLA runtime consists of two cooperating drivers. The user-mode driver (UMD) manages tensor formatting and I/O preparation, while the kernel-mode driver (KMD) encapsulates the functional-unit scheduler and the hardware-abstraction layer, thereby performing resource allocation and command submission. A comprehensive runtime characterization is provided in Veronesi et al. (2020), which systematically evaluates the performance and overhead of NVDLA across diverse CNN workloads. However, the reported measurements were conducted on an FPGA platform with limited main-memory bandwidth and reduced

clock frequency, potentially deviating significantly from ASIC-grade implementations. To enable a meaningful ASIC-oriented comparison, we distill the three dominant runtime operations identified in Veronesi et al. (2020) and summarize them in Table 1.

Network loading is a one-time overhead incurred at the start of an inference session and does not recur across consecutive inferences. In contrast, data preparation—comprising image fetch and preprocessing—is inherent to any AI pipeline, with latency governed by image resolution, main-memory bandwidth, and CPU performance. In high-performance SoC deployments, this stage typically constitutes only a minor fraction of total inference time, and its optimization lies beyond the scope of NPU acceleration; thus, data preparation is excluded from further analysis in this paper. On the other hand, the submit operation is invoked in every inference, and its latency critically governs system throughput and end-to-end inference performance. This operation can be decomposed into operator scheduling and operator execution. While execution time is bounded by the peak capabilities of the hardware engines—offering limited optimization headroom—the scheduling path is entirely software-controlled and remains amenable to improvement. Consequently, this work focuses on enhancing operator-scheduling strategies.

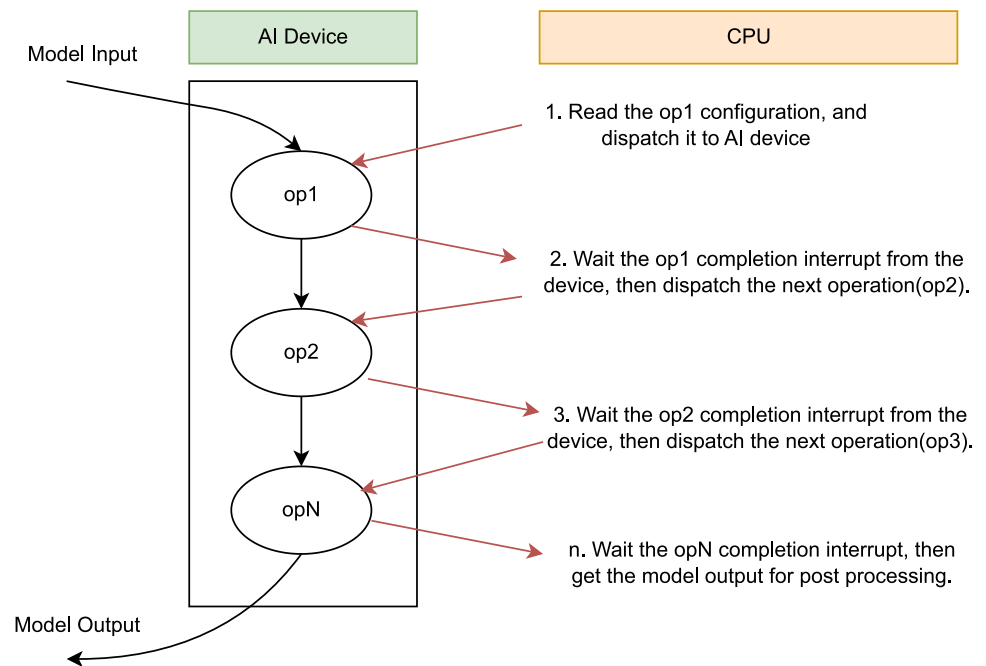
NVDLA's task scheduling typically employs a software-based approach (Nvdla software manual 2017) where the host processor manages operator execution sequencing and dependency resolution. As illustrated in Fig. 2, the scheduling workflow involves runtime software sequentially parsing each operator of AI computing tasks and dispatching them to corresponding execution units (Lyu et al. 2021). This approach provides flexibility and programmability but introduces significant overhead that becomes increasingly problematic as model complexity grows and operator sizes decrease.

However, this conventional approach overlooks the substantial impact of task scheduling mechanisms on system-level efficiency. The operator-level task scheduling relies

Table 1 Runtime operations and performance characteristics for NVDLA inference

Runtime operation	Main actions	Performance factors	Estimated latency
Network loading	Load <code>.loadable</code> file from storage into main memory	Model size DDR bandwidth	~1 ms (ResNet-50)
Data preparation	Image fetch from storage Data process (conversion, normal)	Image size DDR bandwidth CPU performance	~0.1 ms (Image: $224 \times 224 \times 3$)
Submit operation	Schedule per-operation Each operation execution	Operation number CPU-NPU scheduling NPU capability	~5 ms (ResNet-50)

Fig. 2 Software scheduling architecture showing host CPU communication with AI accelerator. The diagram illustrates the frequent interactions between CPU and accelerator required for operator-by-operator scheduling in conventional systems



heavily on CPU-side runtime driver software, revealing two fundamental limitations that become particularly severe in modern neural network architectures:

1. Each operator launch requires CPU-initiated control commands to the accelerator device, involving context switching, driver invocation, and register programming overhead that accumulates across thousands of operators in typical inference tasks.
2. Post-execution operator completion necessitates interrupt-triggered CPU-device state synchronization, causing additional context switches and cache pollution that further degrade overall system performance.

This software-centric scheduling paradigm with frequent CPU-device interactions severely impacts performance, particularly for networks with high-frequency small operators (e.g., LayerNorm in Transformers, activation layers in modern CNNs), where such interactions can consume 20%–40% of total task latency, substantially diminishing the chip's effective computational capability. As neural network architectures continue to evolve toward more complex graphs with finer-grained operators, this scheduling overhead becomes increasingly dominant, limiting the practical performance of even the most powerful computational units.

3 Hardware-managed scheduling architecture

This section provides a comprehensive description of our hardware-managed scheduling system implementation, cov-

ering the architectural principles, hardware design, and software support that enable efficient task scheduling without CPU intervention. Section 3.1 provides a comprehensive system architecture overview and contrasts the workflow differences between software and hardware scheduling approaches, highlighting the key innovations that reduce scheduling overhead. Section 3.2 presents the OCSR mechanism and computation-scheduling instruction set design that form the foundation of our dependency management system. Section 3.3 describes the hardware architecture and parallel execution capabilities that enable out-of-order execution while maintaining correctness. Finally, Section 3.4 outlines the supporting software stack components that bridge the gap between high-level model descriptions and hardware-executable scheduling instructions.

3.1 System architecture overview

Our fully hardware-managed scheduling system builds upon the openNVDLA architecture while fundamentally rearchitecting the scheduling mechanism to eliminate CPU involvement in runtime task management. Unlike openNVDLA's software-based approach, our system implements operator scheduling entirely through hardware-managed computation-scheduling instructions that encode both computational operations and their dependencies in a unified format. As illustrated in Fig. 3, the model deployment workflow begins with compiler optimization, where computational nodes are transformed into hardware-executable operations through a series of graph transformations, operator fusion, and memory layout optimizations. These operations are subsequently seri-

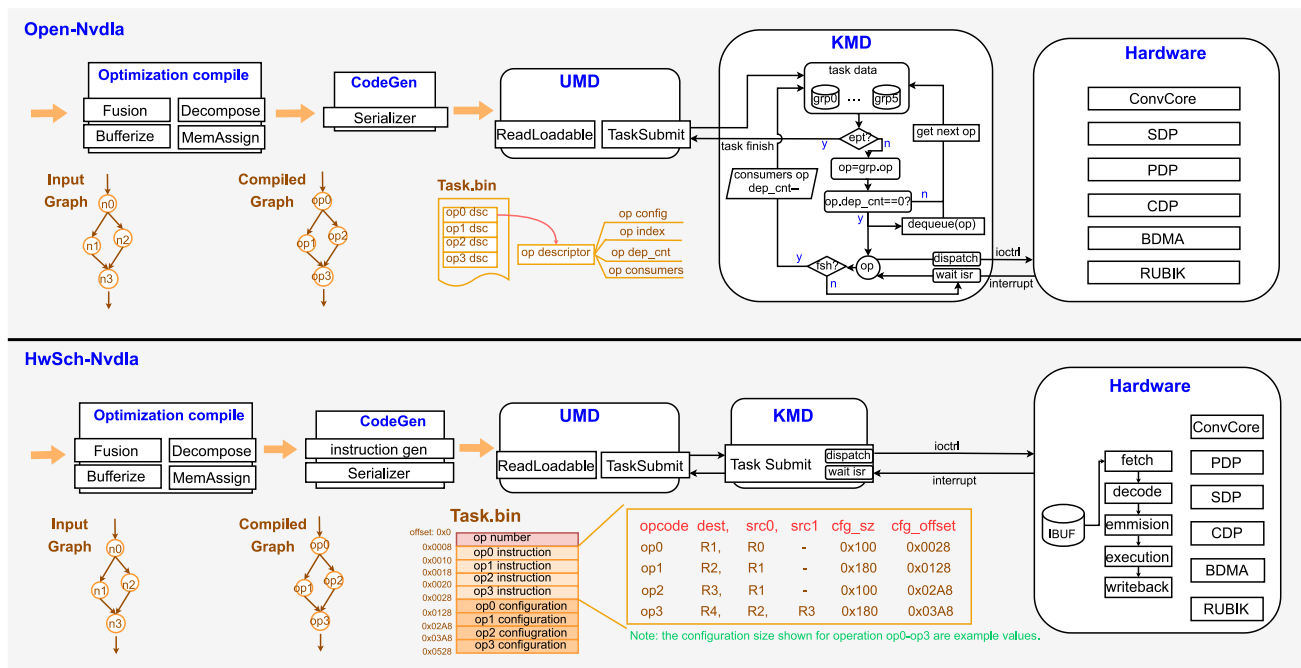


Fig. 3 End-to-end task compilation and execution workflow comparison between openNVDLA and our hardware-scheduled implementation

alized into binary format through a code generator and loaded into hardware via a lightweight runtime software layer that handles only task-level management rather than per-operator control.

While maintaining compatibility with the openNVDLA deployment pipeline to facilitate adoption and comparison, our system introduces a crucial modification at the code generation stage: instead of conventional operation sequences that require runtime interpretation, our compiler maps optimized operations into computation-scheduling instructions directly executable by the hardware scheduler. This transformation occurs during compilation rather than runtime, moving complex dependency analysis and scheduling logic from the critical path of inference execution to offline compilation where computational resources are more abundant and latency constraints are less severe.

3.1.1 OpenNVDLA software scheduling workflow

The conventional software scheduling mechanism in openNVDLA operates through an iterative process that requires continuous CPU involvement:

1. Compiler-generated tasks are parsed by the User Mode Driver (UMD) and forwarded to the Kernel Mode Driver (KMD), which queues operations according to processor types and maintains complex metadata structures for dependency tracking.
2. Each operation is described through a descriptor structure containing `{op_config, op_index, op_dep`

`_cnt, op_consumers}` metadata that must be interpreted by the driver software to determine execution readiness.

3. KMD polls operations in each queue, dispatching dependency-satisfied operations (dependency count = 0) to hardware via IOCTL interfaces that involve costly kernel-user mode transitions and driver overhead.
4. Hardware completion triggers interrupts to KMD, which updates dependency counts for consumer operations and must perform additional scheduling decisions for newly enabled operations.
5. This polling-interrupt cycle continues until all operations complete, with final status returned to UMD through additional mode transitions and data copying operations.

This software-intensive approach creates substantial overhead that grows linearly with the number of operators in the computational graph, making it particularly inefficient for modern neural networks that may contain thousands of small operations.

3.1.2 Proposed hardware scheduling workflow

Our optimized hardware-managed workflow eliminates per-operator CPU involvement through a batch-oriented execution model:

1. Compiler-generated tasks undergo UMD-KMD processing, with KMD delivering complete task packages

directly to the hardware scheduler without interpreting individual operators or their dependencies.

2. Operations are encoded as computation-scheduling instructions, with dependencies managed through OCSRs that are initialized by the compiler and tracked autonomously by hardware.
3. The hardware scheduler autonomously executes instructions through a complete pipeline: fetch-decode-emission-execution-writeback, resolving dependencies dynamically without software intervention.
4. Task completion triggers a single interrupt to KMD, propagating status to UMD through a simplified communication path that involves minimal data transfer and processing.

The hardware-managed approach substantially reduces NPU-CPU communication overhead by limiting interactions to task-level granularity. While software scheduling requires per-operator CPU involvement, our system minimizes interrupt frequency according to the fundamental relationship:

$$\text{Interrupt Reduction} = 1 - \frac{1}{N_{\text{ops}}} \quad (1)$$

where N_{ops} represents operators per task. This optimization significantly enhances scheduling efficiency, particularly for models with numerous small operators where traditional approaches suffer from amortizing fixed overhead costs across minimal computation. For typical neural networks containing hundreds to thousands of operators, this reduction translates to 99.9% fewer interrupts and associated context switching overhead.

As shown in Table 2, the hardware scheduling approach fundamentally changes the scaling characteristics of scheduling overhead, making it constant rather than linear with respect to model complexity. This property becomes increasingly valuable as neural network architectures evolve toward more complex graphs with finer-grained operations.

3.2 OCSR and computation-scheduling instructions

Establishing precise dependencies between operators constitutes a fundamental challenge in AI accelerator scheduling, particularly when enabling parallel execution across heterogeneous computational units. Our solution implements

dependency enforcement through the OCSR mechanism, which provides a lightweight hardware primitive for tracking operator completion states without software intervention. As illustrated in Fig. 4, the workflow comprises three key phases that transform a computational graph into hardware-executable instructions with embedded dependency information:

1. **Per-operator OCSR assignment:** Each computation node receives a dedicated OCSR representing its execution state. Upon completion, the hardware scheduler automatically sets the corresponding OCSR to '1', signaling to dependent operations that their input dependencies have been satisfied. This automatic state update eliminates the need for software tracking and notification mechanisms that consume valuable CPU cycles and introduce latency.
2. **Source-destination dependency chaining:** The OCSR serves as both destination register for current operators and source registers for dependent operators, intrinsically establishing execution ordering through hardware-managed data dependencies. This chaining mechanism naturally captures the producer-consumer relationships in computational graphs without requiring explicit software-managed dependency graphs or complex synchronization primitives.
3. **Computation-scheduling instruction generation:** Each computation node translates into a corresponding instruction with opcode matching operator type, destination register assigned to OCSR, and source registers corresponding to input dependencies. The compiler performs this transformation during code generation, embedding the complete execution schedule directly into the instruction stream rather than relying on runtime decision-making.

The computation-scheduling instructions employ a fixed 64-bit format detailed in Table 3, comprising these key fields that encode both computation and dependency information in a unified representation:

Key instruction components include:

- **Opcode:** 5-bit encoding mapped to specific execution engines, enabling direct dispatch to specialized computational units without runtime resource allocation decisions.
- **Configuration Data:** Extended parameter fields including memory offset and data size that support operations with complex parameter sets exceeding the immediate value capacity of the base instruction format.
- **Source Registers:** Dual 6-bit OCSR addresses with source count indicator that explicitly encode input dependencies, enabling the hardware scheduler to determine execution readiness without software analysis.

Table 2 Communication overhead comparison between scheduling approaches

Scheduling Method	Interrupts/Task	CPU Involvement
Software Scheduling	N_{ops}	Per-operator
Hardware Scheduling	1	Task-level

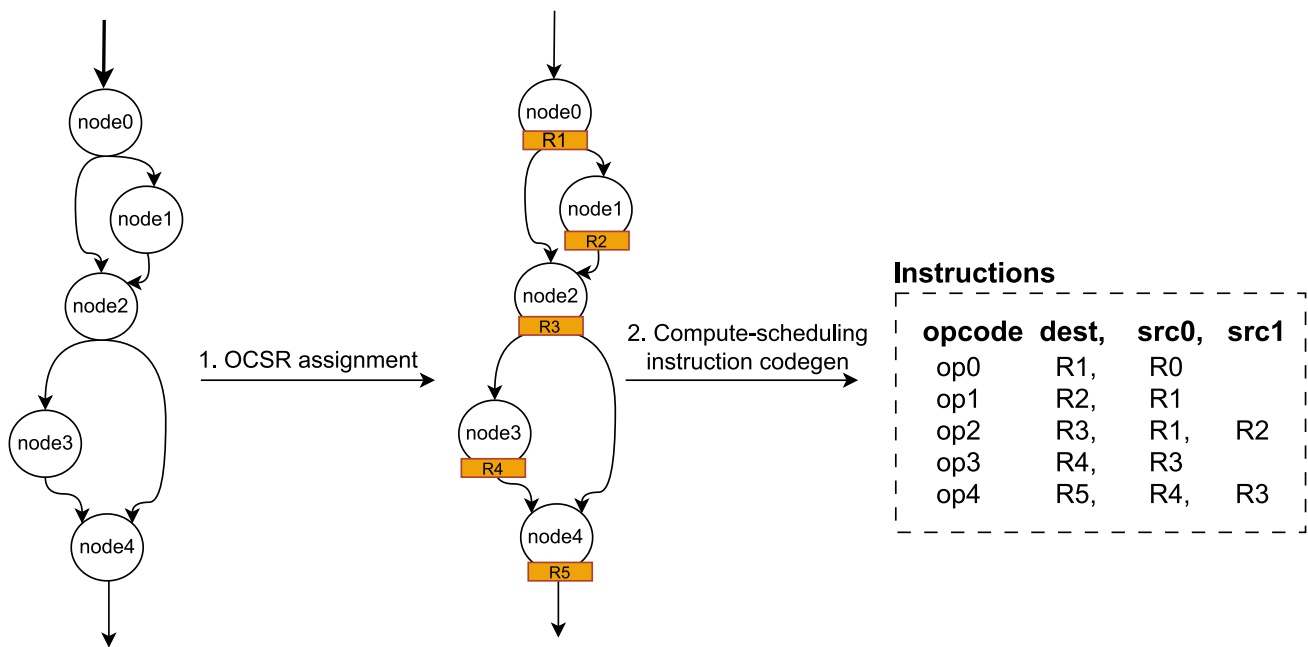


Fig. 4 OCSR assignment and computation-scheduling instruction code generation workflow. The process shows how computational graph nodes are mapped to hardware instructions with explicit dependency encoding through source and destination OCSR specifications

- **Destination Register:** 6-bit OCSR address supporting 64 concurrent operations, providing sufficient parallelism for most practical neural network inference scenarios while maintaining manageable hardware complexity.

This instruction format represents a careful balance between expressive power and hardware complexity, providing sufficient information to encode complex computational graphs while maintaining a fixed-length format that simplifies instruction decoding and pipeline design.

3.3 Hardware scheduler architecture

The proposed hardware architecture, illustrated in Fig. 5, comprises two primary components: a set of neural network execution units and a dedicated hardware scheduling unit. The execution subsystem integrates specialized compute engines inherited from the open NVDLA architecture (e.g., ConvCore, SDP, and PDP), which are responsible for performing neural network computations. These engines retain their original design, with hardware structures fully consis-

Table 3 Computation-scheduling instruction format

Field	Name	Bits	Width	Description
0	Opcode	[63:59]	5	5'h0: illegal 5'h1: ConvCore 5'h2: SDP 5'h3: PDP 5'h4: CDP 5'h5: BDMA
1	Config offset	[58:33]	26	Configuration register address offset for extended operation parameters
2	Config size	[32:23]	10	Configuration data size in bytes for variable-length parameters
3	Source count	[22]	1	Number of source operands (1 or 2) for dependency specification
4	Src1 register	[21:16]	6	Second source OCSR address for multi-input operations
5	Src0 register	[15:10]	6	First source OCSR address for primary input dependency
6	Dest register	[9:4]	6	Destination OCSR address for completion tracking
7	Reserved	[3:0]	4	Reserved for future extensions and special operations

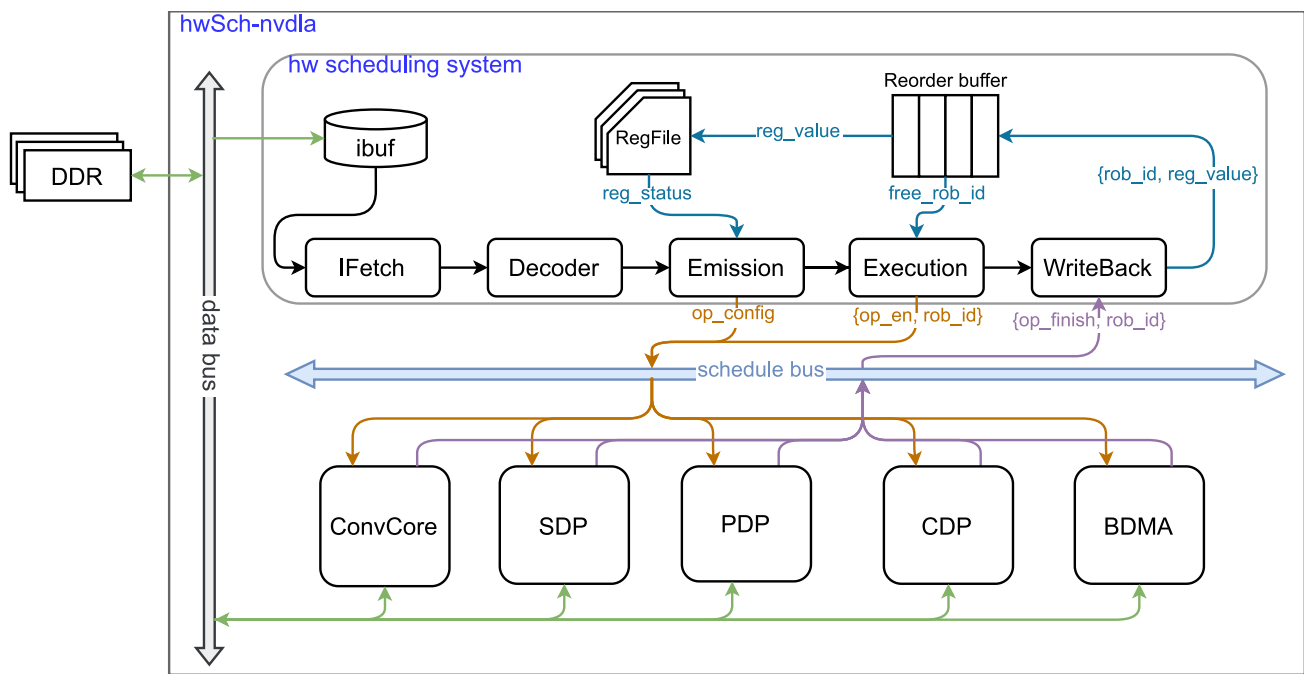


Fig. 5 Hardware architecture integrating dedicated scheduling unit with computational resources. The diagram shows the complete scheduling pipeline alongside specialized execution units

tent with the baseline implementation. Consequently, their microarchitectural details are not elaborated here.

The scheduler implements a complete instruction pipeline that supports fetch, decode, issue, execute, and write-back operations for scheduling instructions. Specifically, the pipeline incorporates an Instruction Buffer (IBUF) for prefetching scheduling instructions and a ReOrder Buffer (ROB) to enable out-of-order execution. The IBUF is sized at 8 KB, designed to hold over 1,000 instructions, which is sufficient for most edge-deployment workloads. Moreover, because each instruction typically occupies the compute engine for an extended duration (thousands to over millions of clock cycles), even a modest IBUF can be replenished from DDR without significantly degrading overall performance. The ROB is configured with 32 entries, an empirically determined size that balances hardware cost and scheduling flexibility. This design is appropriate because AI-operator-level tasks are largely sequential with limited inherent parallelism; furthermore, the long execution time of each instruction on its corresponding engine reduces the marginal performance benefit of a larger ROB.

The scheduling pipeline operates through the following sequence of stages:

1. **Fetch and Decode:** Instructions are retrieved from the instruction buffer and subsequently decoded by the decode module.
2. **Issue:** The emission module issues an instruction once all the following conditions are satisfied: (1) the source

OCSRs are ready, (2) a free slot is available in the reorder buffer (ROB), and (3) the target computational unit is idle. The module then programs the corresponding computational unit with the operation's configurations.

3. **Execution Trigger:** The execution module obtains a free slot (*rob_id*) from the ROB and dispatches it, together with an enable signal, to the appropriate computational unit.
4. **Completion and Writeback:** After execution is completed, the computational unit returns a finish signal along with the *rob_id* to the ROB, writes the result back to the register file, and thereby concludes the scheduling process.

Parallel operator execution presents significant challenges in AI accelerator design due to the complex dependency patterns in neural network computational graphs. Variable execution latencies across operators prevent deterministic completion ordering at compile time, complicating compiler optimizations such as memory reuse and static scheduling. Contemporary systems typically employ sequential scheduling or complex multi-stream mechanisms (Zhang et al. 2021) for limited parallelism, but these approaches either sacrifice performance or introduce substantial software complexity that negates the benefits of hardware acceleration.

Our hardware scheduler, depicted in Fig. 6, enables parallel execution while maintaining dependencies through two key mechanisms:

- **Parallel Issuing:** Simultaneously dispatches dependency-satisfied operators to heterogeneous compute units, max-

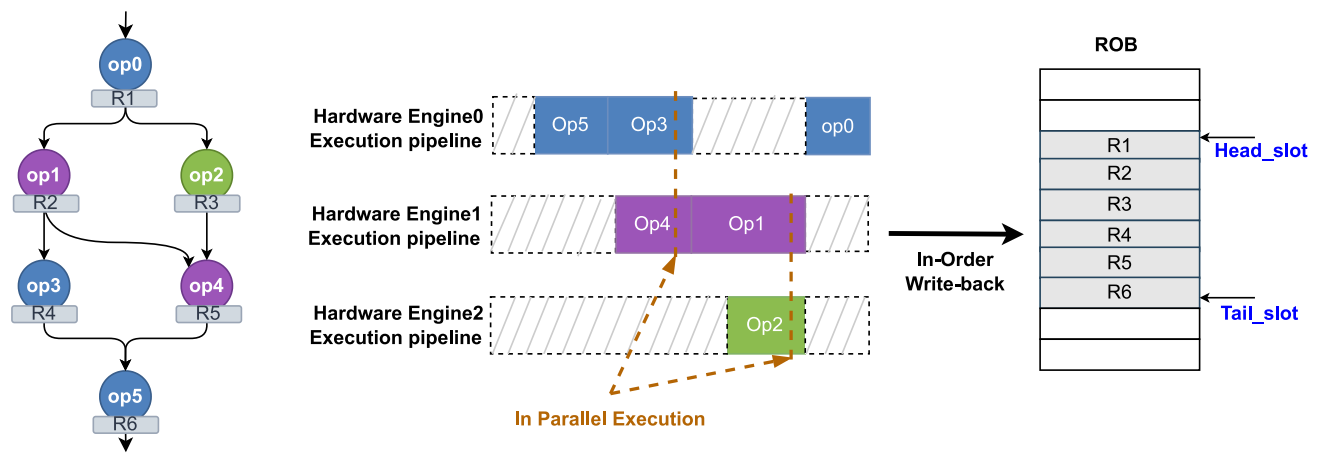


Fig. 6 Out-of-order execution mechanism with in-order writeback maintaining dependency constraints. The diagram illustrates how multiple operators can execute concurrently while the reorder buffer (ROB) ensures correct completion ordering and dependency management

imizing hardware utilization by exploiting independent computation paths within the neural network graph.

- **Reorder Buffer (ROB):** Ensures sequential commit order despite out-of-order execution, maintaining memory consistency and enabling precise interrupt handling for debugging and profiling purposes.

This approach resolves compiler optimization constraints while establishing a scalable paradigm for parallel operator computation. The hardware scheduler dynamically discovers available parallelism during execution rather than relying on static compile-time analysis, adapting to runtime variations in operator execution time and resource availability. This dynamic scheduling capability is particularly valuable for modern neural networks that exhibit data-dependent execution patterns and mixed precision operations with varying computational characteristics.

3.4 Software stack implementation

Our hardware-managed scheduling architecture is supported by a dedicated AI software stack comprising compiler and runtime components that bridge the gap between high-level model descriptions and hardware-executable scheduling instructions. The compiler handles computation graph optimization and instruction generation, transforming abstract computational graphs into concrete scheduling instructions, while the runtime manages task deployment and completion handling with minimal host processor involvement.

3.4.1 Compiler architecture

The compiler incorporates three core functional components that work in concert to generate efficient scheduling instructions:

- **Computation Graph Optimization:** Standard AI compilation techniques including graph fusion, operator splitting, tiling, and memory management that transform high-level model descriptions into hardware-friendly operation sequences. These optimizations improve computational efficiency and resource utilization while maintaining mathematical equivalence with the original model.
- **OCSR Management:** Hardware-scheduled dependency enforcement through OCSR allocation and reclamation that maps abstract data dependencies to concrete register assignments. This component performs liveness analysis and register allocation to maximize OCSR utilization while ensuring correct dependency tracking.
- **Instruction Generation:** Mapping operators to computation-scheduling instructions and generating hardware-executable binaries that encode both computation and scheduling information. This transformation converts optimized operation sequences into the instruction format understood by the hardware scheduler.

The OCSR Management module performs critical register allocation under 64-register constraints, requiring careful resource management to support complex computational graphs within finite hardware resources. The workflow, illustrated in Fig. 7, comprises a three-phase process that ensures efficient register utilization while maintaining correct dependency tracking:

1. **Liveness Analysis:** Per-operator liveness analysis represented as $\{\text{current_node_idx}, \text{last_consumer_node_idx}\}$ that determines the lifetime of each value in the computational graph, enabling precise register allocation and early reclamation.
2. **Register Allocation:** Global free-list management with dynamic register assignment that allocates OCSRs to

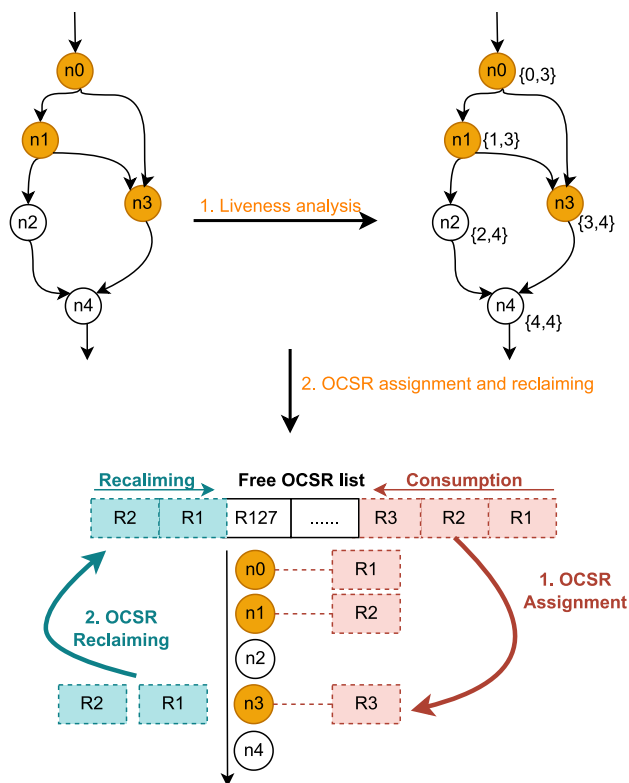


Fig. 7 OCSR allocation and reclamation process during compilation. The diagram shows how registers are allocated to operations based on liveness analysis and reclaimed when no longer needed, enabling efficient use of limited hardware resources

operations based on their liveness properties, minimizing register pressure while avoiding conflicts.

3. **Register Reclamation:** Topology-based liveness analysis with immediate dead register reclamation that returns OCSRs to the free pool as soon as their last consumer has been scheduled, maximizing register reuse across the computational graph.

This compile-time register management eliminates the need for runtime resource allocation, reducing hardware complexity and ensuring deterministic behavior during inference execution. The compiler's ability to precisely manage OCSR lifetimes enables support for complex computational graphs within the constrained register file, making the architecture suitable for modern neural networks with thousands of operations.

3.4.2 Runtime system

Compared to NVDLA's runtime, our implementation achieves substantial complexity reduction by offloading operator scheduling to hardware, transforming the runtime from an active scheduler to a passive manager. The runtime maintains

a dual-layer architecture that separates user-level convenience from kernel-level efficiency:

- **User Mode Driver (UMD):** User interface handling initialization, device management, memory management, and model inference with a focus on API compatibility and ease of integration with existing AI frameworks. The UMD maintains traditional workflows for application compatibility while leveraging the hardware scheduling capabilities for improved performance.
- **Kernel Mode Driver (KMD):** Kernel communication with simplified responsibilities that focus on resource management rather than operation scheduling:
 - Dispatching pre-compiled tasks via IOCTL with minimal parameter validation and setup, leveraging the hardware scheduler for detailed operation management.
 - Handling task completion interrupts and propagating status to user space without intermediate processing or scheduling decisions.

This simplified runtime architecture reduces driver complexity, improves system stability, and minimizes host processor utilization during inference execution. The separation of concerns between hardware scheduling and runtime management creates a more robust and efficient system that can scale to complex models without increasing software complexity or host resource requirements.

4 Experimental evaluation

4.1 Hardware platform and implementation

We present a System-on-Chip (SoC) based on the open-source NVDLA Large core (version 1.0), designed to represent a typical edge AI accelerator. The hardware integrates a dual-core ARM Cortex-A53 CPU, a 2048-MAC array, and a single-channel LPDDR4 interface delivering 25.6 GB/s of bandwidth. As summarized in Table 4, this configuration strikes a balance between substantial computational capability and power efficiency, making it suitable for embedded and edge deployment.

We synthesized the complete SoC design using 28 nm technology to evaluate area efficiency and practical implementation costs. Area analysis in Table 5 shows the hardware scheduler occupies 0.462 mm² (7.41% of total area), representing a modest hardware investment for the substantial performance improvements achieved. Notably, the absolute scheduler area remains constant across die sizes, while its relative percentage decreases significantly in larger

Table 4 Key hardware implementation parameters

CPU	MAC Units	Conv Buffer	Clock	Performance	DDR Bandwidth
ARM-A53 dual-core	2048	512 KB	1 GHz	4 TOPS@int8	25.6 GB/s

designs, making the approach increasingly attractive for high-performance accelerators where area constraints are less severe.

We performed post-synthesis power estimation for the complete SoC running the ResNet-18 workload. Using EDA tools for simulation and for power analysis under typical operating conditions, we obtained the power breakdown summarized for NPU Core in Table 6. The results show that the dedicated hardware scheduling unit (*sched_top*) consumes only 13 mW, representing a mere **1.75%** of the total system power of 741 mW. This minimal overhead confirms the power efficiency of our hardware-managed scheduling architecture.

We evaluated two configurations on hardware emulation platform: reference NVDLA-Large and our hardware-scheduled variant, using identical computational units and memory systems to isolate the impact of scheduling improvements.

4.2 Analysis of scheduling overhead on OpenNVDLA

To isolate and quantify scheduling overhead, we analyze the execution of the YOLOv5s model in openNVDLA. After pruning unsupported operators and partitioning it into 7 compliant subgraphs, we evaluate its performance on the openNVDLA platform, as detailed in Fig. 8. The results reveal fundamental characteristics of software-driven scheduling

- Per-operator software execution time remains consistent at approximately 0.015 ms across subgraphs, reflecting fixed CPU-device interaction overhead that dominates scheduling cost regardless of computational characteristics. This consistency indicates that software scheduling

overhead is largely independent of operator complexity, making it particularly problematic for networks with many small operations.

- Aggregate software overhead constitutes nearly 30% of total execution time in aggregate measurements, with proportionally higher impact in networks featuring smaller operators where computational time decreases while scheduling overhead remains constant. This overhead represents a significant performance loss that cannot be recovered through computational unit optimization alone.

4.3 End-to-end model performance comparison

We further benchmarked ResNet-18, GoogleNetV1, ResNet-50, and YOLOv5s on both platforms to evaluate performance across diverse network architectures and computational patterns. Results in Fig. 9 demonstrate consistent improvements across all tested models:

- Significant performance improvements across all networks, with execution time reduction correlating with software overhead elimination rather than computational optimization. The consistent improvement pattern confirms that our approach specifically targets scheduling overhead without altering fundamental computational capabilities.
- Variation in improvement percentages: ResNet18 (29.4%), GoogleNet (44.5%), ResNet50 (26.4%), YOLOv5s (27.1%) reflects architectural differences in operator composition and dependency structures. Networks with more parallelizable graphs and finer-grained operations show greater benefits from hardware scheduling.
- GoogleNet shows maximum acceleration due to extensive 1x1 convolutions generating numerous small operators where fixed software overhead dominates total

Table 5 Post-synthesis area distribution by module

Module	Area (mm ²)	Percentage
ConvCore	3.863	62.08%
SDP	0.813	13.06%
PDP	0.795	12.78%
Rubik	0.047	0.76%
BDMA	0.029	0.47%
Sched_top	0.462	7.41%
Other	0.214	3.44%
Total	6.223	100%

Table 6 Power and energy efficiency analysis

Metric	Value
NPU Core Total Power (Average)	741 mW
Scheduling Unit (<i>sched_top</i>) Power	15 mW
Scheduling Unit Power Ratio	1.75%
Peak Performance (INT8)	4 TOPS
SoC System Power (Average)	1.87 W
Energy Efficiency	2.14 TOPS/W

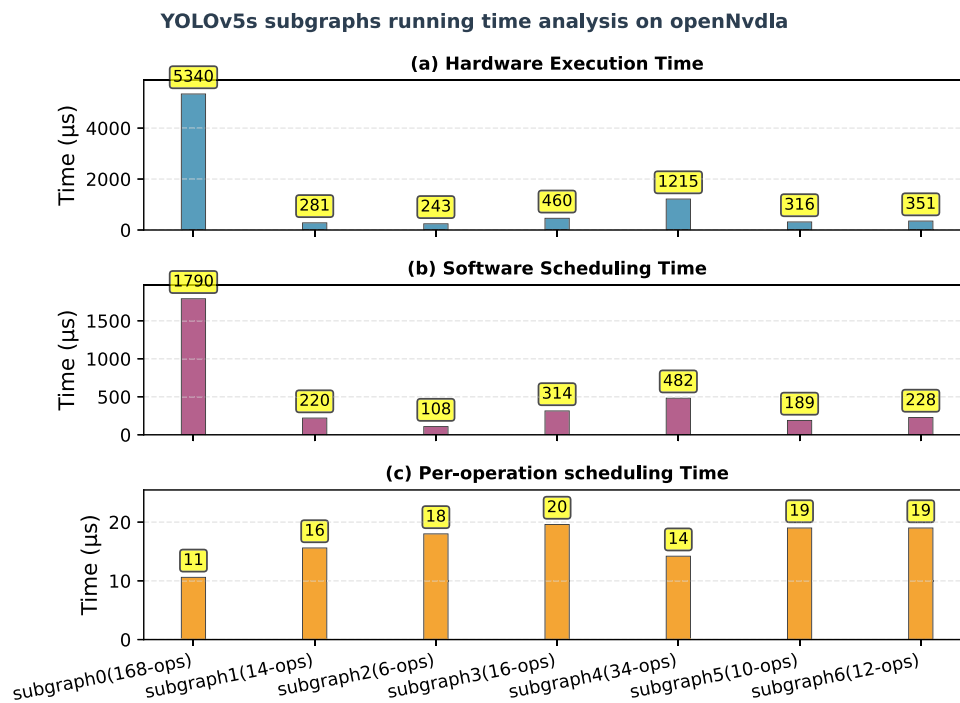


Fig. 8 Performance analysis of YOLOv5s subgraphs execution on openNVDLA platform (averaged over 1,000 independent runs): **(a)** Hardware Execution Time: Accumulated processor computation time of hardware execution. The time varies primarily with workload types and computational sizes. **(b)** Software Scheduling Time: Runtime overhead of the scheduling process. This time correlates with the number of oper-

ators in the subgraph – subgraphs with more operators exhibit longer software scheduling times. **(c)** Per-operation Scheduling Time: Average scheduling time per operator within each subgraph. The nearly constant values across different subgraphs demonstrate that the scheduling time for each operator is deterministic and consistent

execution time. The architecture's high operator count and fine-grained parallelism make it particularly suitable for hardware scheduling approaches.

- Limited benefits from multi-issue parallel execution in tested networks due to sequential structures and data dependencies that limit available parallelism, suggesting

future evaluation with complex models like Transformers that exhibit different computational patterns and potentially greater parallelization opportunities.

The experimental results demonstrate that our hardware-managed scheduling approach effectively addresses the performance limitations of software-based scheduling, particularly for modern neural network architectures that feature numerous small operations and complex dependency patterns. The consistent performance improvements across diverse models highlight the general applicability of our approach, while the minimal area overhead makes it practical for real-world deployment especially in edge scenarios.

The prevalence of Transformer-based models presents a fundamental challenge for the original NVDLA architecture, which is optimized for convolutions and lacks native support for general matrix multiplication (GEMM). Retrofitting GEMM into the openNVDLA framework would require extensive modifications across both the hardware datapath and software scheduling stack, rendering comprehensive performance evaluation infeasible.

In contrast, the proposed hwSch-NVDLA architecture fully offloads task scheduling to dedicated hardware. Consequently, incorporating GEMM support requires only extending the computational datapath, while the software driver

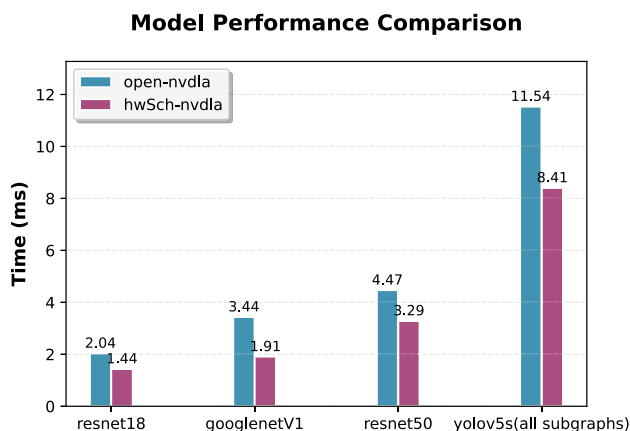


Fig. 9 Performance comparison between openNVDLA and hardware-scheduled implementation across CNN models, based on the average of 1,000 independent runs. The results show consistent performance improvements across diverse network architectures, with particularly significant gains for models with many small operations

Table 7 Performance evaluation of transformer block

Parameter	Value		
Model	Qwen-1.5B transformer block (prefill)		
Token size	256		
Quantized precision	8-bit		
Compiled op number	132		
(a) Test configuration			
	OpenNVDLA (estimated)	HwSch-NVDLA	Speed-up
HW execution time	6.81 ms	6.81 ms	—
SW scheduling time	1.98 ms	—	—
Total time	8.79 ms	7.23 ms	17.75%
(b) Execution time comparison			

remains largely unaffected. To quantify this advantage, we enhanced our hardware design with GEMM capabilities and executed a representative Transformer block from the *Qwen-1.5B* model, recording both total execution time and sub-engine utilization.

The runtime of a correspondingly modified openNVDLA system can be estimated by decomposing the total execution time as:

$$T_{\text{total}} = T_{\text{hw}} + T_{\text{sw}},$$

where T_{hw} is derived directly from hardware execution traces in hwSch-NVDLA, and T_{sw} is estimated by scaling the number of operators with the empirically measured per-operator software scheduling latency. This methodology offers a tractable first-order performance projection without necessitating a full hardware-software co-redesign.

Table 7 compares the execution performance of a transformer block between an estimated OpenNVDLA baseline and our HwSch-NVDLA design. The test configuration employs the prefill stage of a *Qwen-1.5B* (<https://huggingface.co/deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B>) transformer block with a sequence length of 256 tokens, which quantized to 8-bit precision and compiled into 132 operations. The software scheduling time for OpenNVDLA is estimated by scaling the empirically measured per-operation scheduling latency of 0.015 ms by the total

number of compiled operations. Results show that HwSch-NVDLA achieves a total execution time of 7.23 ms, representing a 17.75 % reduction from the estimated 8.79 ms of OpenNVDLA. This improvement stems entirely from the elimination of software scheduling overhead 1.98 ms (132×0.015 ms), while hardware execution time remains identical between the two systems. These findings underscore the advantage of hardware-managed scheduling in compute-intensive transformer workloads.

4.4 Comparison with state-of-the-art AI accelerators

To provide a comprehensive evaluation of our hardware-managed scheduling architecture, we compare it with several state-of-the-art NPUs and AI SoCs in both academic and industrial domains. Table 8 summarizes key architectural features and performance metrics across representative platforms. Our comparison focuses on three critical dimensions: scheduling methodology, energy efficiency, and architectural flexibility.

The comparison reveals a fundamental distinction in scheduling methodologies. Most existing platforms, including Eyeriss v2 (Chen et al. 2019) and Diana (Houshmand et al. 2022), rely on software-based scheduling where the host CPU orchestrates operator execution. This approach offers flexibility but introduces significant communication overhead, particularly for fine-grained operators.

Table 8 Comparison with state-of-the-art AI accelerators

Platform	Year	Process	Interface Granularity	Peak Perf. (TOPS)	Power (W)	TOPS/W
Eyeriss v2 (Chen et al. 2019)	2019	65 nm	Layer level	0.154	–	–
Vega (Rossi et al. 2021)	2021	22 nm	Layer level	0.0322	0.0247	1.3
DIANA (Houshmand et al. 2022)	2022	16 nm	Layer level	–	–	5.56
Marsellus (Conti et al. 2023)	2023	22 nm	Layer level	0.091 (8×8-bit)	0.123	0.74
Our Work	2025	28 nm	Task level	4.0	1.87(SoC)	2.14

In contrast, our architecture implements fully hardware-managed scheduling through dedicated scheduling units and OCSR-based dependency tracking. This approach compiles the entire model into a single task package that can be directly off-loaded from software to hardware, eliminating per-operator CPU intervention entirely and reducing host-processor utilization by 98.2 % compared with software-based alternatives, as measured in our experiments.

In terms of energy efficiency, our SoC achieves 2.14 TOPS/W (Averaging 1.87W @ 4 Tops by executing resnet18), a competitive figure underpinned by the NVDLA's already-efficient computational substrate. While the hardware scheduler itself contributes minimally to this peak efficiency metric, it critically enhances the real-world usable performance by eliminating software bottlenecks, reducing task-launch latency, and improving processing-element utilization. Thus, the primary value of our scheduler lies not in boosting peak TOPS/W, but in maximizing the delivered fraction of that peak performance during actual inference.

Beyond efficiency metrics, our architecture maintains superior flexibility compared to specialized accelerators. The OCSR mechanism and computation-scheduling instruction set enable support for diverse computational patterns beyond conventional CNNs. Unlike fixed-function accelerators optimized for specific operation types (e.g., matrix multiplication in Eyeriss), our scheduling architecture can adapt to emerging neural network structures through compiler mapping rather than hardware redesign.

The comparison with specialized SoCs like Vega (targeting IoT endpoints) and Marsellus (optimized for multi-core RISC-V) demonstrates that our approach achieves competitive efficiency while maintaining broader applicability. The decoupled nature of our scheduling architecture allows it to be integrated with various computational backends, making it suitable for both edge inference and potentially larger-scale deployments through scalability mechanisms.

5 Conclusion and future work

This paper introduces a novel fully hardware-managed scheduling architecture for AI accelerators that addresses the critical performance bottleneck of task scheduling in modern neural network inference. Our contributions include a comprehensive computation-scheduling instruction set that encodes both computation and dependencies in a unified format, a pipelined hardware execution unit that enables out-of-order execution while maintaining correctness, and an optimized software stack that minimizes host processor involvement. The integrated system alleviates CPU-accelerator communication bottlenecks with minimal area overhead (approximately 7%), achieving average perfor-

mance improvements of 30% across diverse CNN workloads by eliminating software scheduling overhead that dominates execution time in many practical scenarios.

The experimental evaluation demonstrates that our approach provides consistent benefits across different neural network architectures, with particularly significant improvements for models featuring numerous small operations where software scheduling overhead constitutes a substantial portion of total execution time. The minimal area overhead and maintained software compatibility make the architecture suitable for practical deployment in modern AI accelerators where computational efficiency and power consumption are critical concerns.

Future work will focus on three primary directions to extend the capabilities and applicability of our hardware-managed scheduling approach:

- **Expanding operator support** for complex models (e.g., Transformers, graph neural networks) that exhibit different computational patterns and dependency structures than conventional CNNs, requiring enhanced scheduling capabilities and potentially extended instruction set features.
- **Enhancing parallel execution capabilities** through more sophisticated dependency analysis and resource management techniques that can exploit finer-grained parallelism in emerging neural network architectures.
- **Exploring dynamic scheduling optimizations** for evolving AI workloads that may benefit from adaptive scheduling policies based on runtime characteristics and quality-of-service requirements.

The hardware-managed scheduling paradigm represents a fundamental shift in AI accelerator design that prioritizes end-to-end efficiency over peak computational performance, addressing the growing importance of system-level optimization in practical AI deployments. As neural network architectures continue to evolve toward more complex and heterogeneous structures, efficient scheduling will become increasingly critical for realizing the full potential of specialized AI hardware.

Acknowledgements The authors would like to thank the technical support team at CETC 58th Research Institute for their valuable assistance in the hardware implementation and testing. We also extend our gratitude to the anonymous reviewers for their insightful comments and suggestions, which have significantly improved the quality of this manuscript.

Author Contributions Libo Cheng: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Writing - Original Draft, Writing - Review & Editing. Liang Yang: Conceptualization, Methodology, Software, Investigation, Writing - Review & Editing. Jian Shao: Software, Validation, Investigation, Data Curation. Xinyi Gu:

Software, Validation, Resources. Rong Qian: Resources, Data Curation. Xinwei Zhang: Resources, Visualization. Donghao Li: Resources, Project administration. Hongbin Wang: Supervision, Project administration. Xiaoqi Xia: Supervision, Project administration.

Data Availability The datasets generated and/or analyzed during the current study are not publicly available due to patent considerations/confidential nature of the hardware design but are available from the corresponding author on reasonable request.

Declarations

Competing interests The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

References

- Chen Y, Luo T, Liu S, Zhang S, He L, Wang J, Li L, Chen T, Xu Z, Sun N et al (2014) Dadiannao: A machine-learning supercomputer. In: 2014 47th Annual IEEE/ACM international symposium on microarchitecture, pp 609–622. IEEE
- Chen T, Moreau T, Jiang Z, Zheng L, Yan E, Shen H, Cowan M, Wang L, Hu Y, Ceze L et al (2018) {TVM}: An automated {End-to-End} optimizing compiler for deep learning. In: 13th USENIX Symposium on operating systems design and implementation (OSDI 18), pp 578–594
- Chen Y-H, Yang T-J, Emer J, Sze V (2019) Eyeriss v2: A flexible accelerator for emerging deep neural networks on mobile devices. *IEEE J Emerg Sel Topics Circ Syst* 9(2):292–308
- Conti F, Paulin G, Garofalo A et al (2023) Marsellus: A heterogeneous risc-v ai-iot end-node soc with 2–8b dnn acceleration and 30%-boost adaptive body biasing. *IEEE J Solid-State Circuits* 59(1):128–142
- Dixon D, Sattar H, Moros N, Kesireddy SR, Ahsan H, Lakkimsetti M, Fatima M, Doshi D, Sadhu K, Hassan MJ et al (2024) Unveiling the influence of ai predictive analytics on patient outcomes: a comprehensive narrative review. *Cureus* 16(5)
- Gholami A, Yao Z, Kim S, Hooper C, Mahoney MW, Keutzer K (2024) Ai and memory wall. *IEEE Micro*
- Golin R, Chelini L, Siemieniuk A, Madhu K, Hasabnis N, Pabst H, Georganas E, Heinecke A (2024) Towards a high-performance ai compiler with upstream mlir. [arXiv:2404.15204](https://arxiv.org/abs/2404.15204)
- Halawani Y, Kilani D, Hassan E, Tesfai H, Saleh H, Mohammad B (2021) Rram-based cam combined with time-domain circuits for hyperdimensional computing. *Sci Rep* 11(1):19848
- Houshmand P, Sarda GM, Jain V et al (2022) Diana: An end-to-end hybrid digital and analog neural network soc for the edge. *IEEE J Solid-State Circuits* 58(1):203–215
- Jouppi NP, Young C, Patil N, Patterson D, Agrawal G, Bajwa R, Bates S, Bhatia S, Boden N, Borchers A et al (2017) In-datacenter performance analysis of a tensor processing unit. In: Proceedings of the 44th annual international symposium on computer architecture, pp 1–12
- Li Z, Wijerathne D, Mitra T (2024) Coarse-grained reconfigurable array (cgra). *Handbook Comput Architect* 465–505
- Liu S, Du Z, Tao J, Han D, Luo T, Xie Y, Chen Y, Chen T (2016) Cambricon: An instruction set architecture for neural networks. *ACM SIGARCH Comput Architect News* 44(3):393–405
- Lu J, Pan B, Chen J, Feng Y, Hu J, Peng Y, Chen W (2024a) Agentlens: Visual analysis for agent behaviors in llm-based autonomous systems. *IEEE Trans Visualiz Comput Graph*
- Lu Q, Zhu L, Xu X, Whittle J, Zowghi D, Jacquet A (2024b) Responsible ai pattern catalogue: A collection of best practices for ai governance and engineering. *ACM Comput Surv* 56(7):1–35
- Lyu X, Zhang W, Wang J (2021) A comprehensive review on the runtime of nvidia deep learning accelerator. In: Proceedings of the 5th international conference on algorithms, computing and systems, pp 88–93
- Malathi D, Gautham S, Dineshkumar M, Balakrishnan K (2024) Design of convolution neural network for crack detection. In: 2024 7th International conference on devices, circuits and systems (ICDCS), pp 60–66. IEEE
- Nvidia hardware architectural specification (2017). <http://nvidia.org/hw/v1/hwarch.html>
- Nvidia software manual (2017). <http://nvidia.org/sw/contents.html>
- Nvidia unit description (2017). http://nvidia.org/hw/v1/ias/unit_description.html
- Rossi D, Conti F, Eggiman M et al (2021) Vega: A ten-core soc for iot endnodes with dnn acceleration and cognitive wake-up from mram-based state-retentive sleep mode. *IEEE J Solid-State Circuits* 57(1):127–139
- Tam E, Jiang S, Duan P, Meng S, Pang Y, Huang C, Han Y, Xie J, Cui Y, Yu J et al (2020) Breaking the memory wall for ai chip with a new dimension. In: 2020 5th South-East Europe design automation, computer engineering, computer networks and social media conference (SEEDA-CECNSM), pp 1–7. IEEE
- Verma N, Jia H, Valavi H, Tang Y, Ozatay M, Chen L-Y, Zhang B, Deaville P (2019) In-memory computing: Advances and prospects. *IEEE Solid-State Circuits Mag* 11(3):43–55
- Veronesi A, Bertozzi D, Krstic M (2020) Assessing the configuration space of the open source nvidia deep learning accelerator on a mainstream mpso platform. In: IFIP/IEEE International conference on very large scale integration-system on a chip, pp 87–112. Springer
- Wang X, Cheng J, Chang F, Zhu L, Chang H, Mei K (2024) A bandwidth enhancement method of vta based on paralleled memory access design. *Integration* 94:102102
- Zhang F, Zhang C, Yang L, Zhang S, He B, Lu W, Du X (2021) Fine-grained multi-query stream processing on integrated architectures. *IEEE Trans Parallel Distrib Syst* 32(9):2303–2320
- Zhou Y, Moosavi-Dezfooli S-M, Cheung N-M, Frossard P (2018) Adaptive quantization for deep neural network. In: Proceedings of the AAAI conference on artificial intelligence, vol 32

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.